SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO3: Analyze relational databases using functional dependencies, normalization (1NF to 5NF), and key constraints to identify redundancy and ensure data integrity. (BL2, BL3)

Activity Tool: Experiments (High)

Assessment Tool Weightage: 40%

QUESTIONS Total Marks: 50

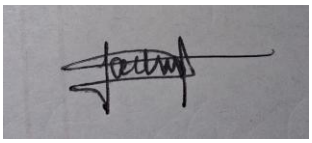
QUESTIONS

Q.No	Question	Bloom's Level	Marks
1	For the relation STUDENT_COURSE(SID, SName, CID, CName, Instructor, Grade), identify all functional dependencies and determine the candidate keys.	BL2 – Understand	5
2	Demonstrate the process of converting an unnormalized relation into 1NF with step-by-step justification and example.	BL3 – Apply	5
3	Given relation R(A,B,C,D,E) with FDs: A→BC, BC→D, D→E. Analyze and decompose it to 2NF eliminating partial dependencies.	BL3 – Apply	5
4	Apply 3NF decomposition on the relation EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID) with transitive dependencies.	BL3 – Apply	5
5	Verify whether R(A,B,C,D) with FDs: AB→C, C→D, D→A is in BCNF. If not, decompose it into BCNF.	BL3 – Apply	5
6	Experimentally verify lossless-join decomposition for a given relation using the tabular (Chase) algorithm.	BL3 – Apply	5
7	Identify the multi-valued dependencies in TEACHER(TID, Subject, Hobby) and decompose it into 4NF.	BL3 – Apply	5
8	Demonstrate how join dependencies lead to redundancy. Decompose a given relation to 5NF (Project-Join Normal Form).	BL3 – Apply	5
9	Given a poorly designed database schema for a college, identify all anomalies (insertion, deletion, update) and normalize it to 3NF.	BL2 – Understand	5
10	Compute the closure of attribute sets for the given FDs and determine all candidate keys for the relation.	BL3 – Apply	5

Code of Conduct

I Jahwanth.G.R-192524384 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

.

Q1.

Definition

A Functional Dependency (FD) is a relationship between attributes where one attribute (or a set of attributes) uniquely determines another attribute. It is represented as $X \rightarrow Y$, where X is the determinant and Y is the dependent attribute.

Given Relation

STUDENT_COURSE(SID, SName, CID, CName, Instructor, Grade)

- SID – Student ID
- SName – Student Name
- CID – Course ID
- CName – Course Name
- Instructor – Course Instructor
- Grade – Student Grade

Step 1: Identify Functional Dependencies

FD1: SID → SName (each Student ID uniquely determines the Student Name)

FD2: CID → CName, Instructor (each Course ID uniquely determines the Course Name and Instructor)

FD3: (SID, CID) → Grade (a grade depends on both the student and the course taken)

Step 2: Determine Candidate Key

Compute the closure of (SID, CID):

$(SID, CID)^+$ = {SID, CID} → apply FD1 → add SName → apply FD2 → add CName, Instructor → apply FD3 → add Grade

$(SID, CID)^+$ = {SID, SName, CID, CName, Instructor, Grade} = all attributes of the relation.

Neither SID alone (SID^+ = {SID, SName}) nor CID alone (CID^+ = {CID, CName, Instructor}) covers all attributes, so (SID, CID) is minimal.

Hence, (SID, CID) is the only candidate key of STUDENT_COURSE. ✓

MySQL Verification

Use an aggregate query to empirically confirm an FD holds: for SID → SName, no SID should map to more than one distinct SName.

```
mysql> CREATE DATABASE co3_lab;
Query OK, 1 row affected (0.724 sec)

mysql> USE co3
ERROR 1049 (42000): Unknown database 'co3'
mysql> USE co3_lab;
Database changed
mysql> CREATE TABLE STUDENT_COURSE (
    ->   SID INT, SName VARCHAR(30), CID VARCHAR(10),
    ->   CName VARCHAR(30), Instructor VARCHAR(30), Grade CHAR(2)
    -> );
Query OK, 0 rows affected (0.821 sec)

mysql> INSERT INTO STUDENT_COURSE VALUES
    ->   (101,'Arun','C1','DBMS','Dr. Rao','A'),
    ->   (101,'Arun','C2','OS','Dr. Kumar','B+'),
    ->   (102,'Divya','C1','DBMS','Dr. Rao','A-');
Query OK, 3 rows affected (0.166 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> -- FD1 check: SID -> SName  (expect 0 rows if FD holds)
Query OK, 0 rows affected (0.007 sec)

mysql> SELECT SID, COUNT(DISTINCT SName) AS distinct_names
    -> FROM STUDENT_COURSE GROUP BY SID HAVING distinct_names > 1;
Empty set (0.170 sec)

mysql> -- FD2 check: CID -> CName, Instructor  (expect 0 rows if FD holds)
Query OK, 0 rows affected (0.007 sec)

mysql> SELECT CID, COUNT(DISTINCT CName) AS c1, COUNT(DISTINCT Instructor) AS c2
    -> FROM STUDENT_COURSE GROUP BY CID HAVING c1 > 1 OR c2 > 1;
Empty set (0.035 sec)
```

Q2.

Definition

First Normal Form (1NF) requires every attribute of a relation to hold a single, atomic value, with no repeating groups or multi-valued attributes in a single cell.

Given Relation (Unnormalized Form – UNF)

STUDENT(SID, SName, {CID, CName}) — a student can enrol in multiple courses, stored as a repeating group inside one row.

SID	SName	Courses (CID, CName)
101	Arun	(C1, DBMS), (C2, OS)
102	Divya	(C1, DBMS)

Step 1: Identify the Violation

The "Courses" attribute holds a set of (CID, CName) pairs inside a single cell for SID 101. This is a repeating group / multi-valued attribute, which violates atomicity — the relation is in UNF, not 1NF.

Step 2: Eliminate the Repeating Group

Each repeating (CID, CName) pair is flattened into its own row, while SID and SName are repeated for every course the student takes. This makes every cell atomic.

Result (1NF)

SID	SName	CID	CName
101	Arun	C1	DBMS
101	Arun	C2	OS
102	Divya	C1	DBMS

Step 3: Justification

- Every attribute now holds a single atomic value — no cell contains a set or list.
- The composite key (SID, CID) uniquely identifies each row.
- Note: 1NF alone does not remove redundancy (SName still repeats) — that is addressed by 2NF/3NF, which is out of scope for this step.

Hence, STUDENT(SID, SName, CID, CName) is in 1NF. ✓

Q3.

Definition

Second Normal Form (2NF) requires the relation to be in 1NF, and every non-prime attribute to be fully functionally dependent on the whole of every candidate key (no partial dependency on a proper subset of a composite key).

Step 1: Determine the Candidate Key

Compute A^+: $A^+ = \{A\} \rightarrow$ apply $A \rightarrow BC \rightarrow \{A,B,C\} \rightarrow$ apply $BC \rightarrow D \rightarrow \{A,B,C,D\} \rightarrow$ apply $D \rightarrow E \rightarrow \{A,B,C,D,E\}$.

$A^+ = \{A,B,C,D,E\}$ = all attributes of R, so A alone is a candidate key. No other single attribute's closure covers all attributes (B, C, D, E never appear as the sole determinant of the rest), and since no FD has A on its right-hand side, A must belong to every candidate key — so {A} is the only candidate key.

Step 2: Check for Partial Dependency

A partial dependency can only exist when the candidate key is composite (more than one attribute) and a non-prime attribute depends on a proper subset of it. Here the candidate key {A} has only one attribute, so it has no proper non-empty subset — partial dependency is structurally impossible.

Verification check: this is a common exam trap — always compute the candidate key first before assuming a multi-attribute FD chain implies a composite key. Here A→BC looks like it might suggest a composite key, but the closure test shows A alone determines everything.

Hence, R(A,B,C,D,E) already satisfies 2NF as given — no decomposition is required to eliminate partial dependency, since the sole candidate key {A} is a single attribute. (Note: R is not in 3NF, because of the transitive chain A→BC→D and D→E; a 3NF decomposition would be a separate step from what this question asks.) ✓

Q4..

Definition

Third Normal Form (3NF) requires the relation to be in 2NF, with no transitive dependency of a non-prime attribute on the candidate key (i.e., no non-prime attribute depends on another non-prime attribute).

Step 1: Identify Functional Dependencies

FD1: EmpID → EmpName, DeptID

FD2: DeptID → DeptName, ManagerID

Step 2: Determine Candidate Key

$EmpID^+$ = {EmpID} → {EmpID, EmpName, DeptID} → apply FD2 → {EmpID, EmpName, DeptID, DeptName, ManagerID} = all attributes. So EmpID is the candidate key.

Step 3: Identify the Transitive Dependency

EmpID → DeptID and DeptID → DeptName, ManagerID, so EmpID → DeptName, ManagerID holds transitively through the non-prime attribute DeptID. Since DeptName and ManagerID are non-prime and depend on DeptID (also non-prime, not a superkey), this violates 3NF.

Step 4: Decompose

Split off the attributes involved in the transitive dependency into their own relation, keyed on the determinant of that dependency:

- EMPLOYEE(EmpID, EmpName, DeptID) — PK: EmpID, FK: DeptID → DEPARTMENT
- DEPARTMENT(DeptID, DeptName, ManagerID) — PK: DeptID

Hence, EMPLOYEE and DEPARTMENT together are in 3NF and preserve all original dependencies. ✓

MySQL Verification

```
mysql> USE co3_lab;
Database changed
mysql> CREATE TABLE DEPARTMENT (
    ->   DeptID VARCHAR(10) PRIMARY KEY,
    ->   DeptName VARCHAR(30),
    ->   ManagerID INT
    -> );
Query OK, 0 rows affected (0.254 sec)

mysql> CREATE TABLE EMPLOYEE (
    ->   EmpID INT PRIMARY KEY,
    ->   EmpName VARCHAR(30),
    ->   DeptID VARCHAR(10),
    ->   FOREIGN KEY (DeptID) REFERENCES DEPARTMENT(DeptID)
    -> );
Query OK, 0 rows affected (0.763 sec)

mysql> INSERT INTO DEPARTMENT VALUES ('D1','CSE',501), ('D2','ECE',502);
Query OK, 2 rows affected (0.094 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql> INSERT INTO EMPLOYEE VALUES (1,'Kavin','D1'), (2,'Priya','D1'), (3,'Ravi','D2');
Query OK, 3 rows affected (0.052 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> -- Rejoin to confirm no information was lost
Query OK, 0 rows affected (0.010 sec)

mysql> SELECT E.EmpID, E.EmpName, E.DeptID, D.DeptName, D.ManagerID
    -> FROM EMPLOYEE E JOIN DEPARTMENT D ON E.DeptID = D.DeptID;
+-------+---------+--------+----------+-----------+
| EmpID | EmpName | DeptID | DeptName | ManagerID |
+-------+---------+--------+----------+-----------+
|     1 | Kavin   | D1     | CSE      |       501 |
|     2 | Priya   | D1     | CSE      |       501 |
|     3 | Ravi    | D2     | ECE      |       502 |
+-------+---------+--------+----------+-----------+
3 rows in set (0.034 sec)
```

Q5.

Definition

Boyce-Codd Normal Form (BCNF) requires that for every non-trivial FD X → Y in the relation, X must be a superkey.

Step 1: Determine All Candidate Keys

AB^+ = {A,B} → apply AB→C → {A,B,C} → apply C→D → {A,B,C,D} = all attributes. AB is a candidate key.

BD^+ = {B,D} → apply D→A → {A,B,D} → now AB→C applies (A and B both present) → {A,B,C,D} = all attributes. BD is also a candidate key.

(A alone, C alone, D alone, AC, AD, CD were checked and none of their closures cover all four attributes.)

Step 2: Test Each FD Against the Candidate Keys

FD	Is LHS a superkey?	BCNF?
AB → C	Yes (AB is a candidate key)	OK
C → D	No (C^+ = {A,C,D}, missing B)	VIOLATES
D → A	No (D^+ = {A,D}, missing B,C)	VIOLATES

Step 3: Decompose (BCNF Algorithm)

Using the violating FD C → D: X = C, X^+ = {A,C,D}. Split into R1 = {A,C,D} and R2 = R – (X^+ – X) = {B,C}.

R1(A,C,D) still has D→A where D is not a superkey of R1 (D^+ = {A,D}, missing C), so decompose again: R1a(A,D) and R1b(C,D).

Final BCNF Decomposition

- R1(A, D) — key D, FD: D → A
- R2(C, D) — key C, FD: C → D
- R3(B, C) — key {B, C}, no non-trivial FD

This decomposition is lossless-join (verified by the BCNF algorithm) but is not dependency-preserving — AB→C cannot be checked without joining all three relations. This trade-off is expected and acceptable in BCNF decomposition.

Hence, R(A,B,C,D) is NOT in BCNF (C→D and D→A both violate it); the lossless BCNF decomposition is {R1(A,D), R2(C,D), R3(B,C)}. ✓

MySQL Verification:

```
mysql> USE co3_lab;
Database changed
mysql> CREATE TABLE R_orig (A INT, B INT, C INT, D INT);
Query OK, 0 rows affected (0.239 sec)

mysql> INSERT INTO R_orig VALUES (1,1,10,100), (2,1,10,100), (3,2,20,200);
Query OK, 3 rows affected (0.086 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> CREATE TABLE R1 AS SELECT DISTINCT A, D FROM R_orig;
Query OK, 3 rows affected (0.251 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> CREATE TABLE R2 AS SELECT DISTINCT C, D FROM R_orig;
Query OK, 2 rows affected (0.202 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql> CREATE TABLE R3 AS SELECT DISTINCT B, C FROM R_orig;
Query OK, 2 rows affected (0.199 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql> -- Rejoin and compare to original row count (lossless check)
Query OK, 0 rows affected (0.003 sec)

mysql> SELECT COUNT(*) AS original_rows FROM R_orig;
+---------------+
| original_rows |
+---------------+
|             3 |
+---------------+
1 row in set (0.079 sec)

mysql> SELECT COUNT(*) AS rejoined_rows FROM
    ->   (SELECT R1.A, R3.B, R2.C, R1.D
    ->    FROM R1 JOIN R2 ON R1.D = R2.D
    ->            JOIN R3 ON R2.C = R3.C) AS rejoined;
+---------------+
| rejoined_rows |
+---------------+
|             3 |
+---------------+
1 row in set (0.017 sec)
```

Q6.

Definition

The Chase (tableau) algorithm tests whether a decomposition {R1, R2} of relation R is lossless-join with respect to a set of FDs. Build a tableau with one row per sub-relation: attributes belonging to that sub-relation get an unsubscripted symbol (a, b, c…), attributes not belonging to it get a subscripted symbol. Repeatedly apply each FD to equate cells; if any row becomes fully unsubscripted, the decomposition is lossless.

Given Relation and Decomposition

R(A,B,C,D) with FDs: A→B, B→C, decomposed into ρ = {R1(A,B,C), R2(A,D)}.

Step 1: Construct the Initial Tableau

	A	B	C	D
R1(A,B,C)	a	b	c	d_1
R2(A,D)	a	b_2	c_2	d

Step 2: Apply the FDs

Apply A→B: both rows agree on column A (both = a), so column B must be equated. Row 1 has the unsubscripted value b, so row 2's b_2 is rewritten to b.

Apply B→C: both rows now agree on column B (both = b), so column C must be equated. Row 1 has the unsubscripted value c, so row 2's c_2 is rewritten to c.

	A	B	C	D
R1(A,B,C)	a	b	c	d_1
R2(A,D)	a	b	c	d

Step 3: Interpret the Result

Row 2 has become fully unsubscripted (a, b, c, d), which means the tableau reduces to a row identical to the original tuple of R — the chase produces a row with no subscripts.

Hence, The decomposition {R1(A,B,C), R2(A,D)} of R(A,B,C,D) is a lossless-join decomposition.
✓

MySQL Verification:

Reproduce the same check experimentally: project sample data into R1 and R2, then verify that joining them reconstructs R exactly.

```
mysql> USE co3_lab;
Database changed
mysql> CREATE TABLE R_chase (A INT, B INT, C INT, D INT);
Query OK, 0 rows affected (0.214 sec)

mysql> INSERT INTO R_chase VALUES (1,10,100,1000), (2,20,200,2000), (3,10,100,3000);
Query OK, 3 rows affected (0.056 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> CREATE TABLE R1_chase AS SELECT DISTINCT A, B, C FROM R_chase;
Query OK, 3 rows affected (0.236 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> CREATE TABLE R2_chase AS SELECT DISTINCT A, D FROM R_chase;
Query OK, 3 rows affected (0.174 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> -- Lossless check: row counts and content must match the original
Query OK, 0 rows affected (0.005 sec)

mysql> SELECT * FROM R_chase ORDER BY A;
+------+------+------+------+
| A    | B    | C    | D    |
+------+------+------+------+
|    1 |   10 |  100 | 1000 |
|    2 |   20 |  200 | 2000 |
|    3 |   10 |  100 | 3000 |
+------+------+------+------+
3 rows in set (0.007 sec)

mysql> SELECT R1_chase.A, R1_chase.B, R1_chase.C, R2_chase.D
    -> FROM R1_chase JOIN R2_chase ON R1_chase.A = R2_chase.A
    -> ORDER BY R1_chase.A;
+------+------+------+------+
| A    | B    | C    | D    |
+------+------+------+------+
|    1 |   10 |  100 | 1000 |
|    2 |   20 |  200 | 2000 |
|    3 |   10 |  100 | 3000 |
+------+------+------+------+
3 rows in set (0.009 sec)
```

Q7.

Definition

A Multi-Valued Dependency (MVD) X →→ Y holds when, for a fixed value of X, the set of Y values is independent of the set of all other attributes (Z). Fourth Normal Form (4NF) requires the relation to be in BCNF and to have no non-trivial MVD unless X is a superkey.

Given Relation Instance

A teacher can teach several subjects and independently have several hobbies — the two facts are unrelated to each other, but storing both in one table forces every subject to be paired with every hobby:

TID	Subject	Hobby
T1	Math	Reading
T1	Math	Painting
T1	Physics	Reading
T1	Physics	Painting

Step 1: Identify the MVDs

TID →→ Subject and TID →→ Hobby

Given a TID, the set of Subjects is independent of the set of Hobbies, which is why every combination of the two must appear — this Cartesian-product pattern is the signature of an MVD, and there is no FD determining Subject or Hobby, so the relation is already in BCNF but not 4NF.

Step 2: Decompose

- R1(TID, Subject)
- R2(TID, Hobby)

Both R1 and R2 have only two attributes, so any MVD in them is trivial by definition — they are automatically in 4NF.

Hence, TEACHER(TID, Subject, Hobby) decomposes losslessly into R1(TID, Subject) and R2(TID, Hobby), eliminating the redundant Subject×Hobby combinations. ✓

MySQL Verification:

```
mysql> USE co3_lab;
Database changed
mysql> CREATE TABLE TEACHER_Subject (TID VARCHAR(10), Subject VARCHAR(20));
Query OK, 0 rows affected (0.216 sec)

mysql> CREATE TABLE TEACHER_Hobby (TID VARCHAR(10), Hobby VARCHAR(20));
Query OK, 0 rows affected (0.205 sec)

mysql> INSERT INTO TEACHER_Subject VALUES ('T1','Math'), ('T1','Physics');
Query OK, 2 rows affected (0.065 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql> INSERT INTO TEACHER_Hobby VALUES ('T1','Reading'), ('T1','Painting');
Query OK, 2 rows affected (0.048 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql> -- Rejoin: this SQL join recreates the Cartesian-product redundancy on purpose,
Query OK, 0 rows affected (0.005 sec)

mysql> -- proving the two facts were independent and the original table was redundant
Query OK, 0 rows affected (0.004 sec)

mysql> SELECT S.TID, S.Subject, H.Hobby
    -> FROM TEACHER_Subject S JOIN TEACHER_Hobby H ON S.TID = H.TID;
+------+---------+----------+
| TID  | Subject | Hobby    |
+------+---------+----------+
| T1   | Physics | Reading  |
| T1   | Math    | Reading  |
| T1   | Physics | Painting |
| T1   | Math    | Painting |
+------+---------+----------+
4 rows in set (0.008 sec)
```

Q8.

Definition

A Join Dependency (JD) *(R1, R2, …, Rn) holds on R if R always equals the natural join of R1, R2, …, Rn. Fifth Normal Form (5NF / PJNF) requires that every non-trivial join dependency is implied by the candidate keys of R — i.e. R cannot be losslessly decomposed any further.

Given Relation (classic Supplier-Part-Project example)

SUPPLY(Supplier, Part, Project), governed by the business rule: "if Supplier S supplies Part P, Part P is used in Project J, and Supplier S supplies to Project J, then S must supply P specifically to J." This three-way circular constraint is a join dependency that cannot be captured by any FD, MVD, or two-way decomposition — the relation may already be in BCNF/4NF and still carry this redundancy.

Step 1: Show the Redundancy

If SUPPLY is kept as one ternary table, adding or changing one fact (e.g. a new Supplier-Part link) forces re-deriving/re-checking the whole table for consistency with the rule above — any single two-attribute projection joined back with itself can reintroduce spurious rows that don't correspond to real facts, unless all three pairwise projections are combined together.

Step 2: Decompose Using the Join Dependency

JD: *(Supplier-Part, Part-Project, Supplier-Project)

- R1(Supplier, Part)
- R2(Part, Project)
- R3(Supplier, Project)

Step 3: Verify

SUPPLY = R1 ⋈ R2 ⋈ R3 (the natural join of all three, taken together, reconstructs the original relation exactly). No pair of these three, joined alone, is guaranteed to reconstruct SUPPLY — all three are required.

Hence, SUPPLY(Supplier, Part, Project) is decomposed into R1(Supplier, Part), R2(Part, Project), R3(Supplier, Project), which is in 5NF. ✓

MySQL Verification:

```
mysql> USE co3_lab;
Database changed
mysql> CREATE TABLE SP (Supplier VARCHAR(10), Part VARCHAR(10));
Query OK, 0 rows affected (0.282 sec)

mysql> CREATE TABLE PJ (Part VARCHAR(10), Project VARCHAR(10));
Query OK, 0 rows affected (0.149 sec)

mysql> CREATE TABLE SJ (Supplier VARCHAR(10), Project VARCHAR(10));
Query OK, 0 rows affected (0.148 sec)

mysql> INSERT INTO SP VALUES ('S1','P1'), ('S1','P2');
Query OK, 2 rows affected (0.102 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql> INSERT INTO PJ VALUES ('P1','J1'), ('P2','J1');
Query OK, 2 rows affected (0.059 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql> INSERT INTO SJ VALUES ('S1','J1');
Query OK, 1 row affected (0.037 sec)

mysql> -- 3-way join reconstructs the original SUPPLY facts
Query OK, 0 rows affected (0.006 sec)

mysql> SELECT SP.Supplier, SP.Part, PJ.Project
    -> FROM SP JOIN PJ ON SP.Part = PJ.Part
    ->         JOIN SJ ON SP.Supplier = SJ.Supplier AND PJ.Project = SJ.Project;
+----------+------+---------+
| Supplier | Part | Project |
+----------+------+---------+
| S1       | P1   | J1      |
| S1       | P2   | J1      |
+----------+------+---------+
2 rows in set (0.011 sec)
```

Q9.

Given Relation (Poorly Designed)

COLLEGE(StudentID, StudentName, DeptID, DeptName, CourseID, CourseName, Grade)

Step 1: Identify Anomalies

- Insertion anomaly: a new Department cannot be recorded until at least one student is enrolled in it (DeptID/DeptName only exist attached to a student row); similarly a new Course cannot be added without a student taking it.
- Deletion anomaly: deleting the last remaining student of a department erases all knowledge of that department (DeptName, etc.) and deleting a student's only course row erases the course's details as a side effect.
- Update anomaly: DeptName is repeated once per enrolled student in that department — changing it requires updating every matching row, and missing even one row leaves the data inconsistent.

Step 2: Identify Functional Dependencies

StudentID → StudentName, DeptID

DeptID → DeptName

CourseID → CourseName

(StudentID, CourseID) → Grade

Step 3: Normalize to 3NF

- STUDENT(StudentID, StudentName, DeptID) — PK: StudentID
- DEPARTMENT(DeptID, DeptName) — PK: DeptID
- COURSE(CourseID, CourseName) — PK: CourseID
- ENROLLMENT(StudentID, CourseID, Grade) — PK: (StudentID, CourseID)

Hence, This 3NF decomposition removes the insertion, deletion, and update anomalies: departments and courses can now be added independently of enrolment, and each fact is stored exactly once. ✓

MySQL Verification:

Demonstrate the anomalies on the original design, then show they disappear after normalization.

```
mysql> USE co3_lab;
Database changed
mysql> CREATE TABLE COLLEGE_bad (
    ->   StudentID INT, StudentName VARCHAR(30), DeptID VARCHAR(10),
    ->   DeptName VARCHAR(30), CourseID VARCHAR(10), CourseName VARCHAR(30), Grade CHAR(2)
    -> );
Query OK, 0 rows affected (0.285 sec)

mysql> INSERT INTO COLLEGE_bad VALUES (1,'Arun','D1','CSE','C1','DBMS','A');
Query OK, 1 row affected (0.051 sec)

mysql> -- Insertion anomaly: cannot add DeptID D2 ('IT') without a student
Query OK, 0 rows affected (0.004 sec)

mysql> -- (would need dummy/NULL StudentID, CourseID values)
Query OK, 0 rows affected (0.004 sec)

mysql>
mysql> -- Update anomaly demo: change DeptName only where it happens to match
Query OK, 0 rows affected (0.003 sec)

mysql> -- (error-prone if a department has many student rows)
Query OK, 0 rows affected (0.003 sec)

mysql> UPDATE COLLEGE_bad SET DeptName = 'Computer Science' WHERE DeptID = 'D1';
Query OK, 1 row affected (0.083 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> -- After normalization, the same rename touches exactly one row:
Query OK, 0 rows affected (0.011 sec)

mysql> CREATE TABLE DEPARTMENT2 (DeptID VARCHAR(10) PRIMARY KEY, DeptName VARCHAR(30));
Query OK, 0 rows affected (0.226 sec)

mysql> INSERT INTO DEPARTMENT2 VALUES ('D1','CSE');
Query OK, 1 row affected (0.043 sec)

mysql> UPDATE DEPARTMENT2 SET DeptName = 'Computer Science' WHERE DeptID = 'D1';
Query OK, 1 row affected (0.032 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

Q10.

Given Relation and FDs

R(A,B,C,D,E) with FDs: A→BC, CD→E, B→D, E→A

Step 1: Compute A^+

A^+ = {A} → A→BC → {A,B,C} → B→D → {A,B,C,D} → CD→E (C and D both present) → {A,B,C,D,E}

A^+ = {A,B,C,D,E} = all attributes → A is a candidate key.

Step 2: Compute E^+

E^+ = {E} → E→A → {A,E} → A→BC → {A,B,C,E} → B→D → {A,B,C,D,E}

E^+ = {A,B,C,D,E} = all attributes → E is also a candidate key.

Step 3: Rule Out Other Single Attributes

Attribute	Closure	Full (=R)?
B	{B,D}	No
C	{C}	No
D	{D}	No

None of B, C, D alone reach all five attributes, and no FD has both A and E derivable from any smaller combination not already tested, so no other minimal key exists.

Hence, R(A,B,C,D,E) has exactly two candidate keys: {A} and {E}. ✓